SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0605 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks: 25**

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

I Dhuneshwaran V / 192324169 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

6. A machine learning model shows the following loss values during training:
(5 Marks)
(a) Epoch 1: Training Loss = 3.0, Validation Loss = 2.7
(b) Epoch 2: Training Loss = 2.6, Validation Loss = 2.3
(c) Epoch 3: Training Loss = 2.3, Validation Loss = 2.1
(d) Epoch 4: Training Loss = 2.1, Validation Loss = 2.0
(e) Epoch 5: Training Loss = 1.9, Validation Loss = 2.2
(f) Epoch 6: Training Loss = 1.7, Validation Loss = 2.5
(g) Epoch 7: Training Loss = 1.6, Validation Loss = 2.8
(h) Epoch 8: Training Loss = 1.4, Validation Loss = 3.1
i) At which epoch does the model achieve optimal generalization?
ii) Identify signs of overfitting from the data.
iii) Recommend two regularization techniques and explain their impact.

Code:

```
#Q1
training_loss = [3.0, 2.6, 2.3, 2.1, 1.9, 1.7, 1.6, 1.4]
validation_loss = [2.7, 2.3, 2.1, 2.0, 2.2, 2.5, 2.8, 3.1]

best_epoch = validation_loss.index(min(validation_loss)) + 1

print("Optimal Generalization Epoch:", best_epoch)

print("\nSigns of Overfitting:")
print("Overfitting starts from Epoch 5 because training loss decreases while validation loss increases.")
```

```
Optimal Generalization Epoch: 4

Signs of Overfitting:
Overfitting starts from Epoch 5 because training loss decreases while validation loss increases.
```

7. In an industrial process, an AI robot is assigned to segregate the front and back tires automatically. Let the number of tires are 250 (125 front and 125 back tires).
(5 Marks)

a. Front identified as Front: 119
a. Front identifies as Back: 06
a. Back identified as Back: 120
a. Back identified as Front: 05
Compute: Accuracy, Precision, Sensitivity, Specificity and F1-Score.

Code:

```python
#Q2
TP = 119
FN = 6
TN = 120
FP = 5

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
f1 = 2 * (precision * recall) / (precision + recall)

print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Sensitivity:", round(recall * 100, 2), "%")
print("Specificity:", round(specificity * 100, 2), "%")
print("F1-Score:", round(f1 * 100, 2), "%")
```

```
Accuracy: 95.6 %
Precision: 95.97 %
Sensitivity: 95.2 %
Specificity: 96.0 %
F1-Score: 95.58 %
```

8. In a medical diagnosis system, an AI model detects whether a patient has a disease or not. Out of 300 cases (150 diseased and 150 healthy): (5 Marks)
a. Diseased identified as Diseased: 138
a. Diseased identified as Healthy: 12
a. Healthy identified as Healthy: 135
a. Healthy identified as Diseased: 15
Compute: Accuracy, Precision, Sensitivity (Recall), Specificity, and F1-Score.

Code:

```python
#Q3
TP = 138
FN = 12
TN = 135
FP = 15

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
f1 = 2 * (precision * recall) / (precision + recall)

print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Sensitivity:", round(recall * 100, 2), "%")
print("Specificity:", round(specificity * 100, 2), "%")
print("F1-Score:", round(f1 * 100, 2), "%")
```

```
Accuracy: 91.0 %
Precision: 90.2 %
Sensitivity: 92.0 %
Specificity: 90.0 %
F1-Score: 91.09 %
```

9. In a spam email filtering system, an AI model classifies emails as Spam or Not Spam. Out of 400 emails (200 spam and 200 non-spam): (5 Marks)
 Spam identified as Spam: 180
 Spam identified as Not Spam: 20
 Not Spam identified as Not Spam: 170
 Not Spam identified as Spam: 30
Compute: Accuracy, Precision, Sensitivity, Specificity, and F1-Score.

Code:

```
#Q4
TP = 180
FN = 20
TN = 170
FP = 30

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
f1 = 2 * (precision * recall) / (precision + recall)

print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Sensitivity:", round(recall * 100, 2), "%")
print("Specificity:", round(specificity * 100, 2), "%")
print("F1-Score:", round(f1 * 100, 2), "%")
```

```
Accuracy: 87.5 %
Precision: 85.71 %
Sensitivity: 90.0 %
Specificity: 85.0 %
F1-Score: 87.8 %
```

10. In a quality inspection system in manufacturing, an AI model identifies defective and non-defective products. Out of 500 products (250 defective and 250 non-defective): (5 Marks)
 Defective identified as Defective: 2 30
 Defective identified as Non-Defective: 20
 Non-Defective identified as Non-Defective: 225
 Non-Defective identified as Defective: 25
Compute: Accuracy, Precision, Sensitivity, Specificity, and F1-Score.

Code:

```
TP = 230
FN = 20
TN = 225
FP = 25

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
f1 = 2 * (precision * recall) / (precision + recall)

print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Sensitivity:", round(recall * 100, 2), "%")
print("Specificity:", round(specificity * 100, 2), "%")
print("F1-Score:", round(f1 * 100, 2), "%")
```

```
Accuracy: 91.0 %
Precision: 90.2 %
Sensitivity: 92.0 %
Specificity: 90.0 %
F1-Score: 91.09 %
```